Proof of concept for integrating TORCS within a DDS environment

Apollonio Marco, Bossi Giacomo

August 14, 2025

Contents

1 Project data

- Project supervisor(s): Prof. Vittorio Zaccaria
- Describe in this table the group that is delivering this project:

Last and first name	Person code	Email address
Marco Apollonio	10764083	marco2.apollonio@mail.polimi.it
Giacomo Bossi	10766073	giacomo3.bossi@mail.polimi.it

- Describe here how development tasks have been subdivided among members of the group, e.g.:
  - Bossi configured the Docker environment to simplify the development process.
  - Bossi configured Qemu emulator to run the ECU that runs FreeRTOS
  - Bossi developed the Uart interrupt to handle the reading of the host keyboard
  - Apollonio worked on the integration of the OpenDDS library within the TORCS simulator.
  - Apollonio developed some high level examples to demonstrate the usage of the DDS communication.
  - Apollonio developed the network configuration for the DDS communication on the FreeRTOS ECU.
- Links to the project source code; Put here, if available, links to public repos hosting your project

2 Project description

2 pages max please

- What is your project about?
- Why it is important for the AOS course?

2.1 Introduction

The project aims to integrate the TORCS simulator within a DDS environment, in order to test the real time communication between a ECU, being it a real one or a simulated one, and the TORCS simulator.

The project is divided in two Proof of Concepts, each one with a specific goal to achieve:

- The first Proof of Concept aims to send the steering commands from the ECU to the TORCS simulator using The Vehicle Signal Specification (VSS) standard. One modification done to the VSS standard is the usage of the steering wheel as an actuator instead of being a sensor.
- The second Proof of Concept aims to create a torque vectoring algorithm on the ECU so that it receives as an input the yaw rate and the lateral acceleration of the car from the simulator and the angle of the steering wheel from the steering wheel. The ECU will then calculate the torque to be applied to each wheel and send it to the simulator. All this communication will be done using the DDS environment, without the usage of the VSS standard.

2.2 Design and implementation

We managed to implement the First Proof of Concept, with the communication between the ECU and the TORCS simulator working as expected.

For the Second Proof of Concept we managed to link the physical engine of the TORCS simulator to where the pilot need to be implemented, but after analyzing the engine code we realized that the to implement the torque vectoring algorithm we need to modify the engine code to calculate all the velocity and acceleration values based on the torque, because as it is now the torque are just for reading purposes.

2.2.1 Design of First Proof of Concept

For the first Proof Of Concept we decided to create one topics in the DDS environment, using the VSS standard for the steering commands:

- SteeringInformation: it's used to send the steering commands to the TORCS simulator.

The ECU will have a Publisher for the SteeringInformation topic. The TORCS simulator will have a Subscriber for the SteeringInformation.

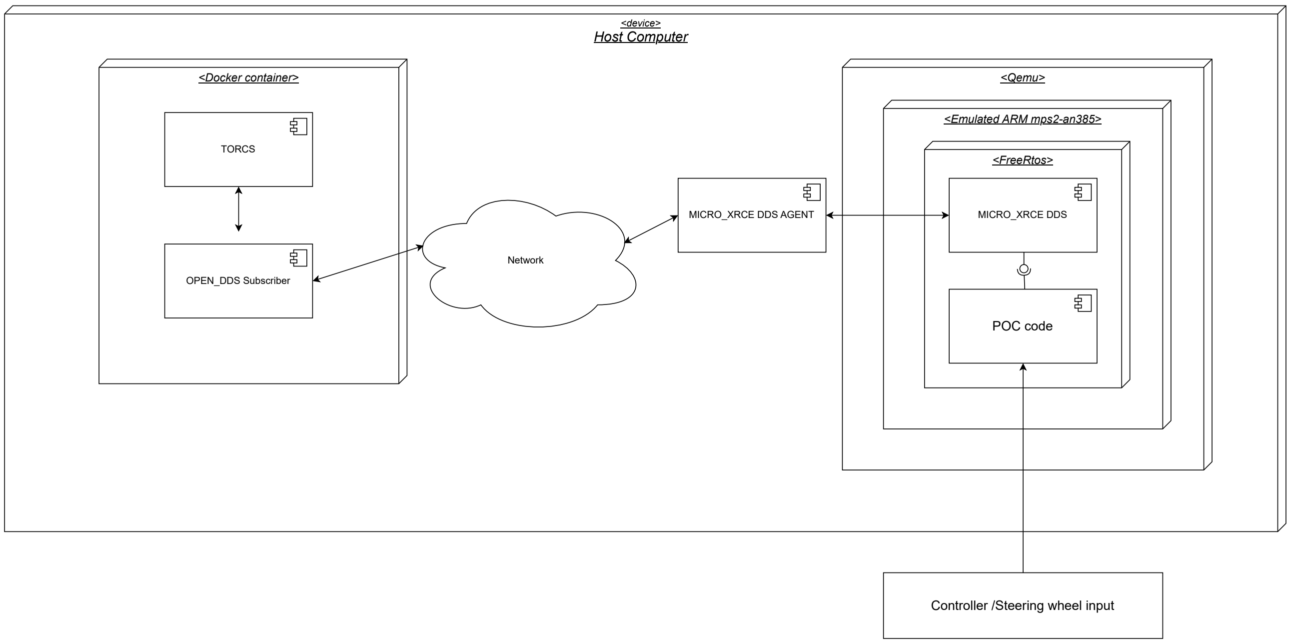

Figure 1: Schematization of the first proof of concept

3 Project outcomes

3.1 Concrete outcomes

Describe the artifacts you've produced, if possible by linking to repo commits. For those who choose to work on an open source project, please put here the **URL to your final pull request**.

Those that have chosen to present a paper can include a link to the slides.

3.1.1 Subscriber Implementation

The implementation of the OpenDDS Subscriber is done in a bot called AOS_dds_hwloop. We implemented specific classes that are used in the bot methods when the race is being created and when the race is started.

When TORCS invokes the method `AOS_dds_hwloop::newrace`, the bot will initialize a static Subscriber object that will do all the configuration needed to create the DDS Domain and Topic to which then it will subscribe and listen to the messages sent by the ECU.

Then when TORCS invokes the method `AOS_dds_hwloop::drive`, the bot will call the method `Subscriber::pop_command` that return the value in the queue, if no message is present then the steering value is the last available one that was received, based on the fact that a steering wheel if left without user input will return to it's base position that is 0 degree.

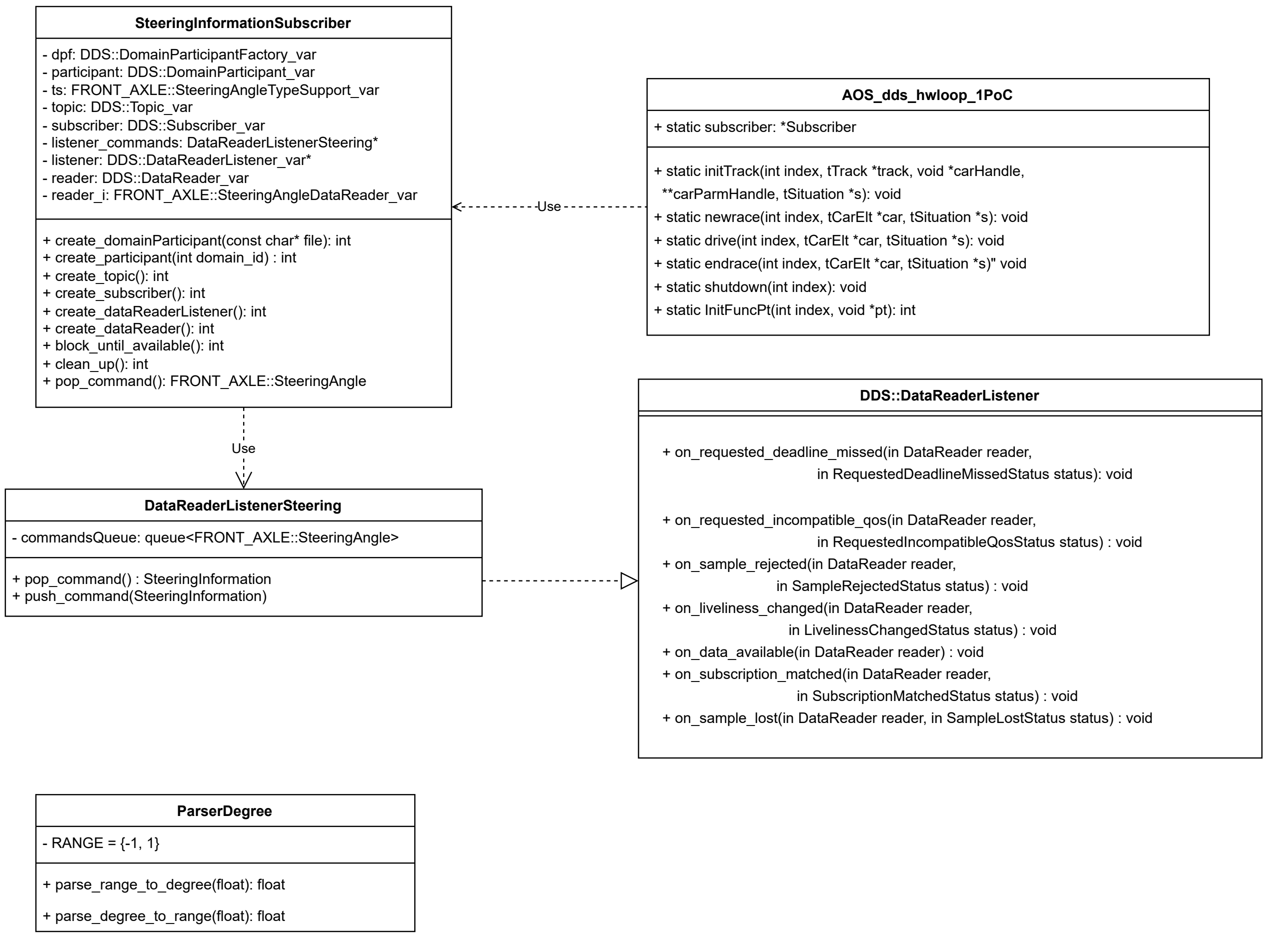

Figure 2: UML diagram of the Subscriber implementation

The topic in the TORCS code has been defined in the file SteeringInformation.idl then to generate the source code from it we created an MPC file that then generated a specific Makefile to create all this files:

- SteeringInformationC.cpp
- SteeringInformationC.h
- SteeringInformationC.inl
- SteeringInformationS.cpp
- SteeringInformationS.h
- SteeringInformationTypeSupport.idl
- SteeringInformationTypeSupportC.cpp
- SteeringInformationTypeSupportC.h
- SteeringInformationTypeSupportC.inl

- SteeringInformationTypeSupportImpl.cpp
- SteeringInformationTypeSupportImpl.h
- SteeringInformationTypeSupportS.cpp
- SteeringInformationTypeSupportS.h

3.1.2 Publisher Implementation and ECU code design

The MCU code has has two functional requirements to fulfill:

- Being able to receive an input from the user and process it to create the appropriate values to add onto the DDS topic.
- Using the DDS library to publish the steering angle data for the TORCS simulator.

In order to receive data we decided to use the UART interface and read from a stream of chars. This is because QEMU offers different ways to expose such interface , like stdin or a tcp port, which makes using the code more flexible. In order to read the stream of chars we decided, given the real-time nature of the application, that a polling driver was a bad idea due to the waste of clock cycles. Avoiding wasted clock cycles is a principle that guided the entire application design, not just the UART interface. As a consequence we decided to use an interrupt-driven approach to handle the incoming data.

The interrupt priority was set to 5, one above the System Call interrupt priority, which is the highest priority that ensures the proper functioning of FreeRTOS ISR safe functions. The interrupt is very simple it just pushes the chars onto a queue so that another task will process them.

About the DDS communication, we decided to use the XRCE-DDS library to handle the communication with the TORCS simulator due to many different reasons:

- The XRCE-DDS library is lightweight and designed for resource-constrained environments, making it suitable for our MCU due to a lower resource usage compared to other DDS implementations.
- The XRCE implementation has direct support to FreeRTOS, which simplifies enormously the integration because other implementations rely on a full POSIX API which FreeRTOS does not have.

To enable DDS we first needed a working TCP/IP stack, so it was necessary to include into the project the library and code of the FreeRTOS Plus TCP along side the XRCE-DDS library. Into the code it was necessary to enable the Ethernet IRQ and set it's priority. This priority is also 5 for the same reason as the UART interrupt. Another thing that needed to be done was the dimension of the stack size of the task that handles the network, it was set as 8 times the minimum stack size. The minimum stack size was set in the FreeRTOSConfigFile as 512. The last thing needed to enable communication was to configure QEMU to do port forwarding between the virtual board and the host machine. As briefly mentioned before the communication should use the VSS standard. So a .idl file obtained from the standard is used to construct the type for the topic.

3.2 Learning outcomes

What was the most important thing all the members have learned while developing this part of the project, what questions remained unanswered, how you will use what you've learned in your everyday life? Please also indicate which tools you learned to use.

Examples:

- Foo learned to write multithreaded applications, he's probably going to create his own startup with what she has learned. She also learned how to debug with gdb.
- Bar learned how to interact with the open source community, politely answering to code reviews and issuing pull requests through Git.

3.3 Existing knowledge

What courses you have followed (not only AOS) did help you in doing this project and why? Do you have any suggestions on improving the AOS course with topics that would have made it easier for you?

3.4 Problems encountered

What were the most important problems and issues you encountered? Did you ever encountered them before?

- Foo encountered a problem with some critical sections. He ended up rewriting existing lock implementation.

4 Honor Pledge

(**This part cannot be modified and it is mandatory to sign it**)

I/We pledge that this work was fully and wholly completed within the criteria established for academic integrity by Politecnico di Milano (Code of Ethics and Conduct) and represents my/our original production, unless otherwise cited.

I/We also understand that this project, if successfully graded, will fulfill part B requirement of the Advanced Operating System course and that it will be considered valid up until the AOS exam of Sept. 2022.

Group Students' signatures